

Multi-Agent Research Assistant

Layane Kanso, Yehya Mazloun, Karim Koiak, Mohamed Darwish
Department of Computer Engineering, Lebanese American University
Course: COE549/749 — Advanced Large Language Models

Abstract—This project implements a research assistant that transforms an initial project idea into a structured brief, a collection of papers and datasets, source assessments, a critical analysis, and an editable report. Five specialist agents coordinate through a LangGraph workflow, while human approval nodes govern research scope and revision. A Pydantic state model, structured model outputs, and in-memory checkpointing provide explicit data contracts and resumable execution. A React interface exposes research evidence, paper reading and comparison, report editing, and saved projects through a FastAPI backend. This report documents implemented architecture, every graph node's state inputs and partial updates, conditional routing, and frontend capabilities. Representative development scenarios are distinguished from independently verified source-level checks. The system supports research preparation; its relevance scores and proposed contributions remain evidence-dependent model judgments.

Index Terms—LangGraph, multi-agent systems, research assistance, Pydantic, human-in-the-loop.

I. INTRODUCTION

Early-stage research involves clarifying a problem, locating relevant literature and usable data, judging their suitability, and identifying a feasible direction. A single conversation can obscure which evidence supports a recommendation and whether essential information is missing.

The application separates these responsibilities into Interviewer, Researcher, Judge, Analyst, and Reporter roles. A shared state preserves the brief and accumulated evidence across stages. Search and inspection tools are assigned to different roles, while explicit user decisions control continuation and revision.

A multi-agent design is appropriate because each stage needs a different prompt, toolset, and output contract: interviewing, source retrieval, source evaluation, critical analysis, and report writing cannot be handled well by a single agent, and keeping them separate lets the Judge check the Researcher's evidence independently and lets human approval gates sit cleanly between stages.

The intended user is a student developing a research or engineering project. The system produces a research-planning report rather than training the proposed model or proving that a proposed contribution is novel. The implementation is a creative application under the course guidelines.

II. SYSTEM ARCHITECTURE

The React/Vite client communicates with FastAPI. The backend runs a LangGraph StateGraph over ResearchState and invokes a configurable Gemini or Claude model through LangChain. Five role-specific agents occupy six AI-processing nodes because reporting and report editing are separate functions. Five additional nodes collect human input, giving 11 graph nodes in total.

A supporting conversational assistant is called through the API rather than registered as a graph node. It answers from project context and translates requests into research, report-editing, or feature-update actions.

A. Main Workflow

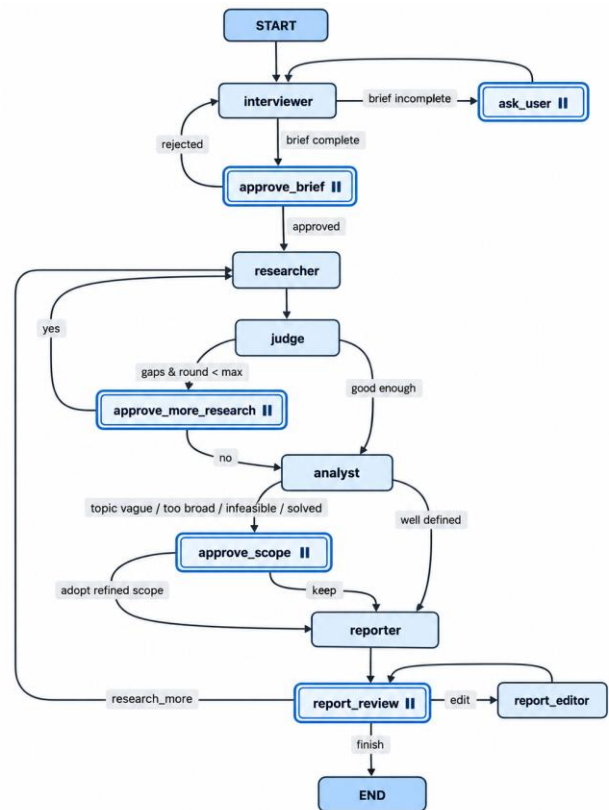


Fig. 1. Main progression. Judge-approved extra searches, adopted scope changes, and report-review research requests return to Researcher. Interview answers/rejection return to Interviewer; report edits return to review. A well-defined topic bypasses scope review.

B. Distinct Toolsets

Researcher: search_kaggle_datasets, search_huggingface_datasets, search_arxiv, and search_semantic_scholar retrieve structured source records and text for the model.

Judge: inspect_kaggle_dataset, inspect_huggingface_dataset, and read_paper collect metadata, previews, and excerpts for evaluation. The tool sets are distinct, satisfying the course requirement.

The shared tool loop binds only the supplied tools, executes model-requested calls, and appends ToolMessage observations. Calls execute sequentially, even when several are requested in one model response.

III. PYDANTIC STATE AND DATA CONTRACTS

ResearchState inherits from pydantic.BaseModel. Table I lists every state field in the reviewed implementation, grouped only to improve readability. Optional values represent information not yet available; lists and dictionaries begin empty unless stated otherwise. The graph uses field-level updates rather than replacing the entire state at every node.

TABLE I. Complete ResearchState field specification

Group	Fields and types	Meaning
Conversation	messages: Annotated[List[AnyMessage], add_messages] idea: str = "" phase: Phase = "interview"	Interview message history, original idea, and current workflow stage. add_messages merges messages by identity.
Interview	interview_turns: int = 0 interview_draft: Optional[DraftBrief] = None brief: Optional[ProjectBrief] = None brief_approved: bool = False	Tracks interview progress, the live draft, the completed brief, and user approval.
Project changes	user_notes: List[str] project_changes: List[str] needs_reevaluation: bool = False	Stores added context and edit history; flags research potentially affected by a changed brief.
Search execution	research_round: int = 0 max_research_rounds: int = 3 search_queries: List[str]	Counts research rounds, limits the Judge-triggered loop, and records previous tool/query combinations.
Sources	datasets: List[Dataset] papers: List[Paper]	Accumulated structured records. Sources are deduplicated by ID within their stored identifiers.
Source judgments	dataset_verdicts: Dict[str, DatasetVerdict] paper_verdicts: Dict[str, PaperVerdict]	Maps source IDs to validity/usefulness or relevance, findings, usage notes, and evidence excerpts.
Coverage	gaps: List[str] gap_details: List[GapDetail] coverage_score: Optional[float] = None	Stores search-focus strings, structured gap explanations, and the model-estimated coverage score.
Judge decisions	needs_more_research: Optional[bool] = None judge_summary: Optional[str] = None more_research_approved: Optional[bool] = None	Separates the Judge's recommendation from the user's approval of another search.
Critical analysis	analysis: Optional[ProblemAnalysis] = None scope_reviewed: bool = False scope_decision: Optional[Literal["adopt", "keep"]] = None	Stores thematic synthesis, feasibility concerns and contribution opportunities, plus scope-review status.
Report	report_markdown: Optional[str] = None report_version: int = 0	Current report and its revision counter. Saved historical report bodies are maintained in session extras.
Review request	pending_instruction: Optional[str] = None pending_section: Optional[str] = None review_action: Optional[Literal["edit", "research_more", "finish"]] = None	Carries requested edits or further research; optionally limits an edit to one report section.
Execution trace	log: Annotated[List[Dict[str, Any]], operator.add]	Appends node event records. The graph wrapper adds timestamps to log entries.

A. Nested Models and Update Semantics

ProjectBrief contains title, goal, domain, problem_type, optional target_variable, features, constraints, research_questions, and keywords. **Dataset** and **Paper** preserve source IDs, URLs, metadata, query provenance, and research round; papers may also reference uploaded PDFs. **ProblemAnalysis** includes themes, concerns, opportunities, refined questions, methodology, and confidence.

Only messages and log use reducers. Other fields are replaced by the values returned in a node update; consequently, Researcher explicitly returns old sources plus new sources, while Judge copies and merges verdict dictionaries. A typical progression is brief=None → approved brief → retrieved sources → verdicts and coverage → analysis → report version 1.

IV. NODE FUNCTIONS, INPUTS, AND PARTIAL UPDATES

Table II covers all 11 registered nodes. Names in the first column identify the actual Python functions (with the common suffix `_node`). Every node returns a partial update dictionary and appends log records; `_stamped` adds a timestamp. The five human nodes additionally merge any notes supplied in the resume payload into `user_notes`. An interrupt itself pauses execution before its subsequent update is returned.

TABLE II. Explicit graph-node contracts

Function	Responsibility	Reads from state	Writes to state (plus log)
<code>interviewer_node</code>	Uses SOM to ask one question or complete the brief.	<code>messages</code> , <code>interview_turns</code> , <code>user_notes</code> ; existing brief if incomplete.	<code>messages</code> ; <code>interview_turns</code> ; <code>interview_draft</code> ; <code>brief</code> ; <code>phase = interview</code> or <code>brief_approval</code> .
<code>ask_user_node</code>	Pauses with <code>interrupt()</code> ; converts the submitted answer into a <code>HumanMessage</code> .	Last <code>messages</code> entry; <code>user_notes</code> .	<code>messages</code> ; optional <code>user_notes</code> .
<code>approve_brief_node</code>	Shows the brief; accepts an edited brief or feedback.	<code>brief</code> ; <code>user_notes</code> .	<code>brief</code> ; <code>brief_approved</code> ; <code>phase = research</code> or <code>interview</code> ; <code>messages</code> on rejection; optional <code>user_notes</code> .
<code>researcher_node</code>	Searches papers/datasets; deduplicates returned source IDs.	<code>brief</code> , <code>user_notes</code> , <code>research_round</code> , <code>gaps</code> , <code>search_queries</code> , <code>datasets</code> , <code>papers</code> .	<code>research_round</code> ; <code>datasets</code> ; <code>papers</code> ; <code>search_queries</code> ; <code>more_research_approved=None</code> ; <code>needs_reevaluation=False</code> ; <code>phase=judging</code> .
<code>judge_node</code>	Inspects promising sources, scores new items in batches, and assesses gaps.	<code>brief</code> , <code>user_notes</code> , <code>datasets</code> , <code>papers</code> , existing verdict dictionaries, <code>research_round</code> .	<code>dataset_verdicts</code> ; <code>paper_verdicts</code> ; <code>gaps</code> ; <code>gap_details</code> ; <code>coverage_score</code> ; <code>needs_more_research</code> ; <code>judge_summary</code> .
<code>approve_more_research_node</code>	Pauses for approval; accepts optional extra search focus.	<code>gaps</code> , <code>coverage_score</code> , <code>research_round</code> , <code>judge_summary</code> , <code>user_notes</code> .	<code>more_research_approved</code> ; <code>gaps</code> ; <code>phase = research</code> or <code>analysis</code> ; optional <code>user_notes</code> .
<code>analyst_node</code>	Produces structured problem analysis and checks whether scope review is needed.	<code>idea</code> , <code>messages</code> , <code>brief</code> , <code>user_notes</code> , <code>judge_summary</code> , <code>coverage_score</code> , <code>gaps</code> , <code>papers/datasets</code> and verdicts, <code>scope_decision</code> , <code>scope_reviewed</code> .	<code>analysis</code> ; <code>phase = scope_review</code> or <code>reporting</code> .
<code>approve_scope_node</code>	Offers adopt/keep for the proposed refined scope.	<code>analysis</code> , <code>brief</code> , <code>user_notes</code> .	<code>scope_reviewed=True</code> ; <code>scope_decision</code> ; <code>phase</code> . On adopt: <code>brief.goal</code> , <code>brief.research_questions</code> , <code>gaps</code> . Optional <code>user_notes</code> .
<code>reporter_node</code>	Writes a thematic report and adds code-generated references and a dataset table.	<code>brief</code> , <code>user_notes</code> , <code>analysis</code> , <code>sources</code> and verdicts, <code>gaps</code> , <code>scope_decision</code> , <code>report_version</code> .	<code>report_markdown</code> ; <code>report_version + 1</code> ; <code>phase=report_review</code> .
<code>report_review_node</code>	Pauses for edit, <code>research_more</code> , or finish, with an optional section/instruction.	<code>report_version</code> , <code>gaps</code> , <code>user_notes</code> .	<code>review_action</code> ; <code>pending_instruction</code> ; <code>pending_section</code> ; <code>gaps</code> and <code>phase=research</code> for more research, or <code>phase=done</code> for finish; optional <code>user_notes</code> .
<code>report_editor_node</code>	Edits the selected section or rewrites the report according to the instruction.	<code>report_markdown</code> , <code>report_version</code> , <code>pending_instruction</code> , <code>pending_section</code> ; same evidence context as Reporter.	<code>report_markdown</code> ; <code>report_version + 1</code> ; <code>pending_instruction=None</code> ; <code>pending_section=None</code> .

Implementation detail: `report_editor` is part of the Reporter role, not a sixth specialist role. `assistant_reply` is an API-invoked helper; human approval nodes are control steps, not independent LLM agents. Some routing steps follow explicit graph edges rather than phase changes: for example, `judge_node` writes its assessment and `route_after_judge` decides the next node.

V. STRUCTURED OUTPUT MODE

The `helper.structured(schema)` calls `get_llm().with_structured_output(schema, method="json_schema")`. The model response is parsed into a Pydantic object rather than treated as arbitrary prose. This supports typed decisions and bounded scores. The following abbreviated schema and invocation reflect the implemented Judge [1].

```
class JudgeReport(BaseModel):
    gaps: List[GapDetail]
    coverage_score: float = Field(
        ge=0, le=1)
    needs_more_research: bool
    summary: str

report = structured(JudgeReport).invoke(
    [SystemMessage(ASSESS_SYSTEM),
     HumanMessage(prompt)])

return {
    "coverage_score": report.coverage_score,
    "needs_more_research":
        report.needs_more_research,
    "gap_details": report.gaps,
    # other partial updates omitted
}
```

Interviewer uses `InterviewTurn`, which contains a reply, `DraftBrief`, completion flag, and optional `ProjectBrief`. Judge uses `VerdictBatch` for item scores and `JudgeReport` for overall coverage. Analyst uses `ProblemAnalysis`; the chat assistant uses `AssistantReply`. Paper metadata, summaries, and comparisons also have typed schemas.

Judge batches contain at most eight candidates. A failed batch is retried once; if both attempts fail, the batch is skipped and logged. Verdict IDs are filtered against known source IDs. Structured validation checks shape and numeric bounds, but does not guarantee factual correctness or complete source coverage.

VI. CONDITIONAL CONTROL FLOW

Seven routing functions implement conditional edges. Interview completion routes to approval or another question; brief approval routes to research or back to Interviewer. The central research-loop condition is:

```
if (state.needs_more_research
    and state.gaps
    and state.research_round
        < state.max_research_rounds):
    return "approve_more_research"
return "analyst"
```

After that approval, yes routes to Researcher and no routes to Analyst. The default limit is three rounds for this Judge-triggered loop. Explicit user research requests and adopted scope changes can start further rounds.

A. Scope and Report Decisions

Analyst sets `phase=scope_review` when the assessment is not well-defined and `scope_reviewed` is false. Otherwise, it proceeds to reporting. The scope gate is therefore offered once unless the state is explicitly reset. Adopting changes the brief's goal and research questions and returns to Researcher; keeping the scope proceeds to Reporter.

Report review maps edit to `report_editor`, `research_more` to `researcher`, and `finish` to `END`. The editor returns to review through a normal edge. These branches make user choices part of graph execution rather than separate UI-only actions.

VII. MEMORY, RESUMPTION, AND STORAGE

The graph compiles with `MemorySaver` and a `JsonPlusSerializer` configured for the nested Pydantic models. A `thread_id` identifies each session. Checkpoints preserve state and execution position during the running process; they also support the history endpoint and retrying a failed node.

Human nodes call `interrupt(payload)` to expose a question or decision. FastAPI resumes the same thread using `Command(resume=value)`. The human function executes again from its start, receives the resume value at the interrupt, and returns its partial update. This is why these nodes keep their main mutation after the interrupt.

Durable project saving is separate from `MemorySaver`. `projects.py` writes state, serialized messages, the next node, chat, pending notes, recent events, and session extras into an atomic JSON file under `backend/data/projects`. The server saves when a graph run pauses, finishes, or fails, and after supported direct edits. It does not persist every intermediate node checkpoint to disk.

Opening a saved project reconstructs `ResearchState` and calls `graph.update_state` with a predecessor node so execution can return to the saved position. A finished project reopens at report review. This restores a current snapshot; the full earlier in-memory checkpoint history is not reconstructed.

A. Activity and Error Handling

Agents emit status, query, result, score, and decision events through LangGraph's custom stream. The server consumes updates and custom events, while the frontend polls session updates. Provider-returned reasoning summaries appear when available; the panel also contains application-generated summaries and outcomes.

Per-session locking prevents overlapping graph starts. Exceptions are exposed as shortened error messages, and a retry endpoint resumes the pending graph work. Reporter retries incomplete output up to three attempts and retains the longest attempt; this is a heuristic, not a correctness guarantee.

VIII. FRONTEND AND USER EXPERIENCE

The React/Vite frontend presents one research workspace with a persistent sidebar, main content area, and contextual panels. Shared API, routing, session, formatting, and research helpers are separated from UI components. Lucide icons, Markdown rendering, reusable primitives, design tokens, and dark-mode styling support a consistent interface [1].

TABLE III. Implemented user-facing components

Area	Components	Behavior
Chat and interview	ChatWorkspace, ChatComposer, InterviewProgress, ApprovalCard, RunSummary, Thinking	Collects the idea and answers; shows a seven-field draft checklist, activity events, agent summaries, PDF upload, and inline approval decisions.
Project overview	ProjectOverview	Displays and edits the brief and Analyst assessment. Changes after research set a needs-re-evaluation flag and preserve an edit history.
Research library	ResearchLibrary, Cards, DatasetDetail	Searches, filters, and sorts papers/datasets by source, status, feature, and year; exposes source links, dataset details, and verdicts.
Paper reading	PaperReader, PaperSummary, SourceChat	Displays extracted text or a known source PDF, structured summaries, Judge notes, and passage-specific Explain/Summarize/Ask actions. Falls back to abstract/source link.
Paper comparison	PaperComparison	Compares 2-5 papers using structured fields for objectives, methods, models, datasets, hardware, results, limitations, and relevance; supports follow-up questions.
Gaps and evidence	ResearchGaps, EvidenceMap	Relates project features to papers, datasets, and gaps; shows evidence and contribution opportunities. Research-this-gap actions use an eligible workflow decision point.
Report workspace	ReportWorkspace	Supports full or section-level AI editing, manual changes, historical versions and restoration; exports PDF, Word, Markdown, and references as BibTeX.
History and settings	HistoryTimeline, Settings, sidebar/project controls	Shows milestones, raw logs/checkpoints, model/source configuration, and saved project selection. Projects can be reopened or deleted.

IX. BACKEND SERVICES AND REPORT ASSEMBLY

FastAPI services. Session endpoints create, inspect, resume, retry, and chat with a project. Additional endpoints update the brief; read, upload, evaluate, summarize, and compare papers; answer source-specific questions; edit/restore/export reports; expose history and configuration; and manage saved projects. Graph execution runs in a background thread while the frontend retrieves progress.

Paper access. library.py caches PDFs and extracted text from stored source links or uploaded files. Uploads are checked for a PDF signature and a 30 MB size limit. Reader extraction is limited to 60 pages; model context is truncated separately. This is distinct from the Judge's shorter inspection tool. Scanned PDFs without extractable text are not handled by an OCR pipeline.

Report production. Analyst synthesizes prior work into themes and assesses scope, feasibility, data realism, and contribution opportunities. Reporter uses this analysis and source evidence to write seven sections: problem, prior work, critical assessment, gaps/contributions, recommended direction, candidate datasets, and references. material.py assigns reference numbers and builds the source table in code. The report editor replaces one matched section or edits the full report; exports are generated by export.py.

Configuration and launch. llm.py selects Gemini or Claude through environment variables; source credentials are configured separately. start.bat/start.sh install missing dependencies, build the frontend, and launch the application. cli.py offers a terminal workflow that prints node updates. The default local application address is `http://localhost:8000`.

X. REPRESENTATIVE SCENARIOS AND VALIDATION

This is a creative application, so validation focuses on observable workflow behavior rather than a benchmark score. The development record describes live runs and browser checks, but those reports are not an independently reproduced evaluation of the final commit. Source inspection confirmed 11 nodes and seven conditional routing functions, and all 24 Python files passed a syntax parse. No end-to-end API run or quantitative accuracy study was performed for this report.

Scenario	Interaction	Working behavior / evidence
Clarify and approve	Reject the first brief, give feedback, then approve a revised brief.	Returns to Interviewer, retains messages, and starts research only after approval. Recorded in the crop-yield development run [3].
Fill missing evidence	Judge identifies gaps; user accepts or declines another round.	Accept returns to Researcher with gap-focused queries; decline advances to Analyst. Both paths are described in the development record [3].
Refine a broad idea	Input: "Use AI to fix traffic in cities."	Recorded assessment: too_broad. Adopting the proposed narrower scope triggered further research. A scoped Arabizi sentiment example was reported as well_defined [3].
Continue and revise	Reopen a saved interview or edit one report section.	Development record describes successful restart/resumption and section-only edits [3]. These are representative observations, not aggregate reliability metrics.

XI. LIMITATIONS AND FUTURE WORK

Evidence depth and confidence. Judge inspects selected sources: Kaggle metadata/files, Hugging Face previews, and up to six PDF pages capped at 7,000 characters. Other judgements may use search metadata or abstracts. Relevance, coverage, confidence, and novelty labels are model estimates. The synthetic-data score cap is prompt-based. UI acceptance begins at 0.7, while report evidence includes scores of at least 0.3; inclusion does not imply UI acceptance. Missing evidence does not prove a field-wide research gap.

State consistency. Existing verdicts are normally skipped by Judge, so a changed brief can leave old scores in use; Researcher clears needs_reevaluation before those records are reassessed. Adopting a scope changes only goal and research_questions, so other brief fields can remain inconsistent. A future revision should invalidate affected verdicts and synchronize the complete brief. Source IDs also do not guarantee cross-provider paper deduplication.

Operational scope. Search access, rate limits, model latency, PDF extraction, and schema failures affect completion. Agents and tools execute sequentially. Saved snapshots do not provide complete durable checkpoint history, and no automated test suite was identified. Useful extensions include citation/quote verification, broader data auditing, source-aware re-evaluation, regression tests for all branches, and measured relevance and completion-rate evaluation.

XII. REQUIREMENTS COVERAGE AND CONCLUSION

The implementation satisfies the structural course requirements in source: five agent roles and different Researcher/Judge toolsets (Section II); Pydantic state with non-string and Optional fields (III); explicit node inputs and partial writes (IV); SOM schemas and invocation (V); seven conditional routers (VI); MemorySaver and human resumption (VII). Frontend and supporting services are covered in VIII-IX, and representative scenarios in X. The system provides a coordinated, inspectable research workflow while retaining user control over scope, additional searches, and report revisions.